UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA
FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

CARRERA:

SOFTWARE.

TEMA:

“Gestor de proyectos con EDT y Kanban”

AUTOR:

Franco Reyes Bryan Cristopher

Marcos Joel Altamirano
Victor Izaguirre

DOCENTE:

ING. ANTHONY ABRAHAN PACHAY ESPINOZA

LA LIBERTAD –

ECUADOR

2026

DOCUMENTACIÓN DEL PROYECTO: GESTOR DE PROYECTOS CON EDT Y KANBAN

1. Resumen Ejecutivo

El **Gestor de Proyectos con EDT y Kanban** es una aplicación web local diseñada para planificar el trabajo de un proyecto, descomponerlo en entregables y tareas mediante una **Estructura de Desglose del Trabajo (EDT / WBS)**, asignar responsabilidades y visualizar el avance del flujo de trabajo en tiempo real a través de un **Tablero Kanban** interactivo.

El proyecto implementa de manera estricta el paradigma de **Programación Funcional**, separando limpiamente la lógica pura de negocio (*Functional Core*) de los efectos externos y la interfaz de usuario (*Imperative Shell*).

2. Arquitectura del Sistema (Functional Core / Imperative Shell)

La arquitectura se divide en dos capas claramente delimitadas:

- **Functional Core (Núcleo Funcional):**
    - Ubicado en el directorio /app/domain/.
    - Contiene la lógica de negocio pura, exenta de efectos secundarios (side-effects). No realiza operaciones directas de E/S ni manipula bases de datos.
    - Implementa estructuras de datos inmutables, funciones de orden superior, lambdas, closures, pattern matching y manejo funcional de errores mediante mónadas (Result).
- **Imperative Shell (Capa Imperativa):**
    - Ubicado en /app/infrastructure/ y /app/ui/.
    - Gestiona los efectos externos, la persistencia local en **SQLite**, el enrutamiento y la interfaz de usuario interactiva construida con **ReactPy** e integrada a través de **FastAPI**.

3. Evidencia de Conceptos de Programación Funcional

Para el cumplimiento de los lineamientos de la asignatura, el proyecto demuestra el uso de los siguientes temas:

1. **Inmutabilidad y Funciones Puras:** Uso de estructuras congeladas (dataclass(frozen=True)) y tuplas. Ninguna función modifica datos en el lugar (*in-place*); todas retornan nuevas instancias con las transformaciones aplicadas.
2. **Manejo Funcional de Errores (Mónada Result):** Implementación de una estructura de tipos Result (Success / Failure) para validar entradas y reglas de negocio de manera segura sin recurrir a excepciones descontroladas o valores None ambiguos.
3. **Pattern Matching:** Uso de la sintaxis match ... case de Python para manejar de forma declarativa las transiciones de estado de las tareas en el tablero Kanban.
4. **Funciones de Orden Superior, Lambdas y Closures:** Procesamiento de colecciones mediante map, filter, reduce y funciones creadas dinámicamente mediante *closures* para filtros de usuarios y entregables.

5. **Decoradores:** Aplicación de decoradores para el registro y auditoría de transformaciones puras en el flujo de datos.

4. Distribución de Responsabilidades y Contribuciones del Equipo

Para garantizar la verificabilidad de la participación de todos los integrantes en el repositorio Git:

Integrante	**Rol Principal / Área de Desarrollo**	**Tareas Clave y Evidencias en Repositorio**
Integrante 1	Líder de Arquitectura y Dominio WBS	• Diseño de modelos inmutables (models.py). • Funciones puras para manipulación de la EDT (logic.py).
Integrante 2	Especialista en Lógica Funcional y Errores	• Implementación de la mónada Result (results.py). • Pattern matching para transiciones Kanban y pruebas unitarias (test_logic.py).
Integrante 3	Desarrollador de Persistencia (Shell)	• Conexión y gestión de la base de datos local SQLite (database.py). • Mapeo entre capas y manejo de efectos externos.
Integrante 4	Desarrollador de Interfaz de Usuario	• Construcción de componentes interactivos con ReactPy (components.py). • Integración del servidor FastAPI y renderizado web (main.py).